

ARCADE: Adversarially Regularized Convolutional Autoencoder for Network Anomaly Detection

William Tessaro Lunardi^{1b}, *Member, IEEE*, Martin Andreoni Lopez^{1b}, *Member, IEEE*, and Jean-Pierre Giacalone

Abstract—As the number of heterogeneous IP-connected devices and traffic volume increase, so does the potential for security breaches. The undetected exploitation of these breaches can bring severe cybersecurity and privacy risks. Anomaly-based Intrusion Detection Systems (IDSs) play an essential role in network security. In this paper, we present a practical unsupervised anomaly-based deep learning detection system called ARCADE (Adversarially Regularized Convolutional Autoencoder for unsupervised network anomaly DETection). With a convolutional Autoencoder (AE), ARCADE automatically builds a profile of the normal traffic using a subset of raw bytes of a few initial packets of network flows so that potential network anomalies and intrusions can be efficiently detected before they cause more damage to the network. ARCADE is trained exclusively on normal traffic. An adversarial training strategy is proposed to regularize and decrease the AE's capabilities to reconstruct network flows that are out-of-the-normal distribution, thereby improving its anomaly detection capabilities. The proposed approach is more effective than state-of-the-art deep learning approaches for network anomaly detection. Even when examining only two initial packets of a network flow, ARCADE can effectively detect malware infection and network attacks. ARCADE presents 20 times fewer parameters than baselines, achieving significantly faster detection speed and reaction time.

Index Terms—Unsupervised anomaly detection, autoencoder, generative adversarial networks, automatic feature extraction, deep learning, cybersecurity.

I. INTRODUCTION

THE PROLIFERATION of IP-connected devices is skyrocketing and is predicted to surpass three times the world's population by 2023 [1]. As the number of connected devices increases and 5G technologies become more ubiquitous and efficient, network traffic volume will follow suit. This accelerated growth raises overwhelming security concerns due to the exchange of vast amounts of sensitive information through resource-constrained devices and over untrusted heterogeneous technologies and communication protocols. Advanced security controls and analysis must be applied to maintain a sustainable, reliable, and secure cyberspace. IDSs play an essential role in network security, allowing for detecting and responding to potential intrusions

and suspicious activities by monitoring network traffic. IDSs can be implemented as signature-based, anomaly-based, or hybrid. Signature-based IDSs detect intrusions by comparing monitored behaviors with pre-defined intrusion patterns, while anomaly-based IDSs focus on knowing normal behavior to identify any deviation [2].

The vast majority of existing network IDSs are based on the assumption that traffic signatures from known attacks can be gathered so that new traffic can be compared to these signatures for detection. Despite high detection capabilities for known attacks, signature-based approaches cannot detect novel attacks since they can only detect attacks for which a signature was previously created. Regular database maintenance cycles must be performed to add novel signatures for threats as they are discovered. Acquiring labeled malicious samples, however, can be extremely difficult or impossible to obtain. The definition of signature-based IDSs, or any other supervised approach for the task, becomes even more challenging when the known class imbalance problem is faced while dealing with public network traffic datasets is considered. Network traffic datasets are known for being highly imbalanced towards examples of normality (non-anomalous/non-malicious) [3] while lacking in examples of abnormality (anomalous/malicious) and offering only partial coverage of all possibilities can encompass this latter class [4].

In contrast, anomaly-based IDSs relies on building a profile of the normal traffic. These systems attempt to estimate the expected behavior of the network to be protected and generate anomaly alerts whenever a divergence between a given observation and the known normality distribution exceeds a pre-defined threshold. Anomaly-based IDSs do not require a recurrent update of databases to detect novel attack variants, and their main drawback usually is the False Alarm Rate (FAR), as it is challenging to find the boundary between the normal and abnormal profiles. These approaches have gained popularity in recent years due to the explosion of attack variants [5], [6], which relates to their ability to detect previously unknown or zero-day threats. Additionally, they do not suffer from the dataset imbalance problem since it only requires normal traffic during training.

Deep Learning (DL) has emerged as a game-changer to help automatically build network profiles through feature learning. It can effectively learn structured and complex non-linear traffic feature representations directly from the raw bytes of a large volume of normal data. Based on a well-represented traffic profile, it is expected that the system's capabilities for isolating anomalies from the normal traffic will increase while

Manuscript received 30 April 2022; revised 19 November 2022 and 25 November 2022; accepted 13 December 2022. Date of publication 16 December 2022; date of current version 6 July 2023. The associate editor coordinating the review of this article and approving it for publication was K. El-Khatib. (Corresponding author: William Tessaro Lunardi.)

The authors are with the Secure System Research Center, Technology and Innovation Institute, Abu Dhabi, UAE (e-mail: willian@ssrc.tii.ae; martin@ssrc.tii.ae; jean-pierre@ssrc.tii.ae).

Digital Object Identifier 10.1109/TNSM.2022.3229706

decreasing the FAR. However, the naive adoption of DL may lead to misleading design choices and the introduction of several drawbacks, such as slow detection and reaction time. In addition to carefully defining the model's architecture, training artifices could be exploited to improve the method's effectiveness without degrading the efficiency due to the increased number of parameters and model size.

In this paper, we propose ARCADE, an unsupervised DL approach for network anomaly detection that automatically builds a profile of the normal traffic (training exclusively on the normal traffic) using a subset of bytes of a few initial packets of network traffic flow as input data. It allows early attack detection preventing any further damage to the network security while mitigating any unforeseen downtime and interruption. The network traffic can be originated from real-time packet sniffing over a network interface card or from a .pcap file. The proposed approach combines two deep neural networks during training: (i) an AE trained to encode and decode (reconstruct) normal traffic; (ii) a critic trained to provide high score values for real normal traffic samples, and low scores values for their reconstructions. An adversarial training strategy is settled where the critic's knowledge regarding the normal traffic distribution is used to regularize the AE, decreasing its potential to reconstruct anomalies, addressing the known generalization problem [7], [8], [9], where (in some scenarios) anomalies are reconstructed as well as normal samples. During detection, the error between the input traffic sample and its reconstruction is used as an anomaly score, i.e., traffic samples with high reconstruction error are considered more likely to be anomalous. The significant contributions of this paper are summarized as follows:

- An unsupervised DL-based approach for early anomaly detection that automatically builds the network traffic profile based on the raw packet bytes of network flows of the normal traffic. It can detect (novel) network anomalies given a few initial packets of network flows, allowing it to prevent network attacks before they could cause further damage.
- A Wasserstein Generative Adversarial Networks with Gradient Penalty (WGAN-GP) adversarial training to regularize AEs, decrease its generalization capabilities towards out-of-the-normal distribution samples, and improve its anomaly detection capabilities.
- A compact convolutional AE model inspired by Deep Convolutional GAN (DCGAN) [10]. The model presents higher accuracy, faster reaction time, 20 times fewer parameters than baselines.
- An extensive validation of ARCADE conducted on several network traffic datasets to assess its capabilities in detecting anomalous traffic of several types of malware and attacks.

The remainder of the paper is laid out as follows: Section II provides the necessary background for Generative Adversarial Networks (GANs). Section III reviews and discusses previous relevant works in the field of DL for anomaly detection and network anomaly detection. Section IV describes the proposed network flows preprocessing pipeline, model architecture, loss functions, and adversarial training strategy. Section V presents the ablation studies and experimental analysis and comparison

of ARCADE's effectiveness and complexity with respect to the considered baselines. Finally, Section VI concludes this paper.

II. BACKGROUND

A. Generative Adversarial Networks

The GANs [11] framework establishes a min-max adversarial game between a generative model G and a discriminative model D . The discriminator $D(\mathbf{x})$ computes the probability that a point $\mathbf{x}$ in data space is a sample from the data distribution rather than a sample from our generative model. The generator $G(\mathbf{z})$ maps samples $\mathbf{z}$ from the prior $p(\mathbf{z})$ to the data space. $G(\mathbf{z})$ is trained to maximally confuse the discriminator into believing that the samples it generates come from the data distribution. The process is iterated, leading to the famous minimax game [11] between generator G and discriminator D

$$\min_G \max_D \mathbb{E}_{\mathbf{x} \sim \mathbb{P}_r} \log(D(\mathbf{x})) + \mathbb{E}_{\tilde{\mathbf{x}} \sim \mathbb{P}_g} \log(1 - D(\tilde{\mathbf{x}})), \quad (1)$$

where $\mathbb{P}_r$ is the data distribution and $\mathbb{P}_g$ is the model distribution implicitly defined by $\tilde{\mathbf{x}} = G(\mathbf{z})$, where $\mathbf{z} \sim p(\mathbf{z})$ is the noise drawn from an arbitrary prior distribution.

Suppose the discriminator is trained to optimality before each generator parameter update. In that case, minimizing the value function amounts to minimizing the Jensen-Shannon divergence (JSD) between $\mathbb{P}_r$ and $\mathbb{P}_g$ [11], but doing so often leads to vanishing gradients as the discriminator saturates [12], [13].

B. Wasserstein Generative Adversarial Networks

To overcome the undesirable JSD behavior, Arjovsky et al. [12] proposed Wasserstein Generative Adversarial Networks (WGAN) that leverages Wasserstein distance $W(q, p)$ to produce a value function that has better theoretical properties than the original. They modified the discriminator to emit an unconstrained real number (score) rather than a probability. In this context, the discriminator is now called a critic. The min-max WGAN training objective is given by

$$\min_G \max_C \mathbb{E}_{\mathbf{x} \sim \mathbb{P}_r} [C(\mathbf{x})] - \mathbb{E}_{\tilde{\mathbf{x}} \sim \mathbb{P}_g} [C(\tilde{\mathbf{x}})]. \quad (2)$$

When the critic C is Lipschitz smooth, this approach approximately minimizes the Wasserstein-1 distance $W(\mathbb{P}_r, \mathbb{P}_g)$. To enforce Lipschitz smoothness, the weights of C are clipped to lie within a compact space $[-c, c]$. However, as described in [12], weight clipping is a terrible approach to enforcing the Lipschitz constraint.

Gulrajani et al. [13] proposed an alternative approach where a soft version of the constraint is enforced with a penalty on the gradient norm for random samples $\hat{\mathbf{x}} \sim \mathbb{P}_{\hat{\mathbf{x}}}$. When considering the WGAN-GP proposed in [13], the critic's loss is given by

$$\mathbb{E}_{\mathbf{x} \sim \mathbb{P}_r} [C(\mathbf{x})] - \mathbb{E}_{\tilde{\mathbf{x}} \sim \mathbb{P}_g} [C(\tilde{\mathbf{x}})] + \lambda_C \mathcal{L}_{GP}, \quad (3)$$

where λ_C is the penalty coefficient, and

$$\mathcal{L}_{GP} = \mathbb{E}_{\hat{\mathbf{x}} \sim \mathbb{P}_{\hat{\mathbf{x}}}} [(\|\nabla_{\hat{\mathbf{x}}} C(\hat{\mathbf{x}})\|_2 - 1)^2], \quad (4)$$

where $\mathbb{P}_{\hat{\mathbf{x}}}$ is the distribution defined by the following sampling process: $\mathbf{x} \sim \mathbb{P}_r$, $\tilde{\mathbf{x}} \sim \mathbb{P}_g$, $\alpha \sim U(0, 1)$, and $\hat{\mathbf{x}} = \alpha \mathbf{x} + (1 - \alpha) \tilde{\mathbf{x}}$.

III. RELATED WORK

Herein, we discuss the relevant works employing DL for anomaly detection. We first present DL anomaly detection approaches that have emerged as leading methodologies in the field of image and video. Then, we comprehensively analyze these novel DL methods and their potential application to network anomaly detection. We categorize unsupervised anomaly detection methods into generative models or pre-trained networks, introduced in Sections III-A and III-B, respectively. Finally, Section III-C presents the DL-related works for network traffic classification and baselines for unsupervised anomaly detection.

A. Generative-Based Anomaly Detection

Generative models, such as AEs [6], [23] and GANs [11], [12], [13], can generate samples from the manifold of the training data. Anomaly detection approaches using these models are based on the idea that anomalies cannot be generated since they do not exist in the training set.

AEs are neural networks that attempt to learn the identity function while having an intermediate representation of reduced dimension (or some sparsity regularization) serving as a bottleneck to induce the network to extract salient features from some dataset. These approaches aim to learn some low-dimensional feature representation space on which normal data instances can be well reconstructed. The heuristic for using these techniques in anomaly detection is that, since the model is trained only on normal data, normal instances are expected to be better reconstructed from the latent space than anomalies. Thus, the distance between the input data and its reconstruction can be used as an anomaly score. Although AEs have been successfully applied across many anomaly detection tasks, in some cases, they fail due to their strong generalization capabilities [7], i.e., sometimes anomalies can be reconstructed as well as normal samples. Bergmann et al. [23] shows that AEs using the Structural Similarity Index Measure (SSIM) [24] can outperform complex architectures that rely on a per-pixel value ℓ_2 -loss. Gong et al. [8] tackle the generalization problem by employing memory modules which can be seen as a discretized latent space. Zhai et al. [9] connect regularized AEs with energy-based models to model the data distribution and classify samples with high energy as an anomaly.

GAN-based approaches assume that only positive samples can be generated. These approaches generally aim to learn a latent feature space of a generative network so that the latent space well captures the normality underlying the given data [25]. Some residual between the real and generated instances is then defined as an anomaly score. One of the early GAN-based methods for anomaly detection is AnoGAN [26]. The fundamental intuition is that given any data instance $\mathbf{x}$; it aims to search for an instance $\mathbf{z}$ in the learned latent features space of the generative network G so that the corresponding generated instance $G(\mathbf{z})$ and $\mathbf{x}$ are as similar as possible. Since the latent space is enforced to capture the underlying distribution of training data, anomalies are expected to be less likely to have highly similar generated counterparts than normal instances. One main issue with AnoGAN is

the computational inefficiency, which can be addressed by adding an extra network that learns the mapping from data instances onto latent space, i.e., an inverse of the generator, resulting in methods like EBGAN [27]. Akcay et al. [28] proposed GANomaly that further improves the generator over the previous works by changing the generator to an encoder-decoder-encoder network. The AE is trained to minimize a per-pixel value loss, whereas the second encoder is trained to reconstruct the latent codes produced by the first encoder. The latent reconstruction error is used as an anomaly score.

The idea behind AEs is straightforward and can be defined under different Artificial Neural Network (ANN) architectures. Several authors have already investigated the applicability of AEs for network anomaly detection [5], [6]. However, its naive adoption can lead to unsatisfactory performance due to its vulnerability to noise in the training data and its generalization capabilities. We propose an adversarial regularization strategy with a carefully designed and compact AE parameterized by 1D-Convolutional Neural Network (CNN). The adversarial training is employed to deal with the aforementioned AE's weaknesses. Similarly to GANomaly, our approach employs an adversarial penalty term to the AE to enforce it to produce normal-like samples. Therefore, we also consider the GANomaly framework as a baseline and compare it with the proposed ARCADE for network anomaly detection.

B. Pretrained-Based Anomaly Detection

Pretrained-based anomaly detection methods use models trained on large datasets, such as ImageNet, to extract features [29]. These pre-trained models produce separable semantic embeddings and, as a result, enable the detection of anomalies by using simple scoring methods such as k -Nearest Neighbor (k -NN) or Gaussian Mixture Models [30]. Surprisingly, the embeddings produced by these algorithms lead to good results even on datasets that are drastically different from the pretraining ones. Recently, [31] showed that using a k -Nearest Neighbor for anomaly detection as a scoring method on the extracted features of a pre-trained ResNet model trained on the ImageNet produces highly effective and general anomaly detection methods on images. That alone surpassed almost all unsupervised and self-supervised methods. In [32], it is shown that fine-tuning the model using either center loss or contrasting learning leads to even better results.

The application of those methods for network anomaly detection is challenging primarily due to the detection's complexity related to the additional required scoring step. Even with a compact model, such as the proposed in Section IV-B with 184k parameters, or the EfficientNet B0 with 5.3M parameters, the requirement for a post-processing scoring procedure makes it unsuitable for online detection, e.g., after forwarding the sample through the model for feature extraction, computing the anomaly score for a given sample's feature vector with k -Nearest Neighbor as the scoring method (as proposed in [31]), implies $O(nl)$ time complexity, where n is the number of training samples, and l is the length of the feature vectors. These techniques appear unexplored and may stand out for *offline* network anomaly detection.

TABLE I
DEEP LEARNING RELATED WORKS FOR NETWORK INTRUSION DETECTION. FOR WORKS THAT USED RAW NETWORK TRAFFIC AS INPUT, WHEN SPECIFIED, WE PRESENT THE NUMBER OF PACKETS (N) AND BYTES (L) USED AS INPUT

Paper	UD ¹	AT ²	RT ³	Granularity	Input Data	Architecture
Vu et al. [14]	○	●	○	Session	Hand-designed features	Augmented dataset with AC-GAN
Truong-Huu et al. [5]	●	●	○	Flow/Session	Hand-designed features	MLP GAN with AE Generator
Doriguzzi-Corin et al. [15]	○	○	○	Flow	Hand-designed features	1D-CNN
Wang et al. [16]	○	○	●	Flow	All layers, $l = [600, 800]$	2D-CNN and LSTM
Wang et al. [17]	○	○	●	Flow/Session	All/L7 layers, $l = 784$	2D-CNN
Yu et al. [18]	○	○	●	Session	All layers, $l = 1000$	Dilated 2D-CNN
Wang et al. [19]	○	○	●	Flow/Session	All/L7 layers, $l = 784$	1D-CNN
Aceto et al. [20]	○	○	●	Session	All/L7 layers, $n \in [4, 32]$, $l \in [256, 2304]$	1D/2D-CNN, LSTM
Lotfollahi et al. [21]	○	○	●	Packet	IP packet, $l = 1024$	2D-CNN
Hwang et al. [6] ⁴	●	○	●	Flow	All layers, $n \in [2, 5]$, $l \in [40, 80]$	1D-CNN with MLP AE
Ahmad et al. [22]	○	○	●	Session	All layers, $n \in [1, 3]$, $l = 450$	1D-CNN
<i>This paper</i>	●	●	●	Flow/Session	All layers, $n \in [2, 5]$, $l = 100$	1D-CNN GAN with AE Generator

¹ Unsupervised anomaly detection ² Adversarial Training ³ Raw Traffic ⁴ Baseline

TABLE II
ISCX-IDS DATASET

Normal		Anomaly	
Traffic Type	# of Flows	Traffic Type	# of Flows
HTTP, SMTP, SSH, IMAP, POP3, and FTP	869,978	Infiltration	9,925
		HTTP DoS	3,427
		DDoS	21,129
		Brute Force SSH	6,964

TABLE III
USTC-TFC DATASET

Normal			Anomaly	
App	Traffic Type	# of Flows	Traffic Type	# of Flows
Bittorrent	P2P	15,00	Cridex	24,581
Facetime	Voice/Video	6,000	Geodo	47,666
FTP	Data transfer	202,034	Htbot	12,652
Gmail	Email/Webmail	17,178	Miuref	20,755
MySQL	Database	172,114	Neris	44,605
Outlook	Email/Webmail	14,984	Nsis-ay	11,014
Skype	Chat/IM	12,000	Shifu	15,766
SMB	Data transfer	77,781	Tinba	16,208
Weibo	Social Network	79,810	Virut	58,638
World of Warcraft	Game	15,761	Zeus	21,741

C. Deep Learning for Network Traffic Classification

Several works have studied DL network traffic classification under the supervised setting. Few works also have studied adversarial training strategies for network traffic classification based on hand-designed features. Nonetheless, feature learning-based unsupervised network anomaly detection with adversarial training appears currently unexplored. Table I summarizes our related works, which are categorized into: (i) Unsupervised anomaly detection (UD) when only normal traffic is considered at the training stage. (ii) Adversarial Training (AT) when GAN-based strategies are applied during training. (iii) Raw Traffic (RT) when the input is the raw network traffic. When RT is considered, the considered protocol layers, the number of initial bytes l , and the number of packets n are presented.

Most deep learning-based traffic classification and anomaly detection approaches rely on feature engineering. We highlight

a few works in which adversarial training or unsupervised anomaly detection was addressed and which rely on hand-designed features. Vu et al. [14] proposed the use of a GANs for dealing with the imbalanced data problem in network traffic classification. The synthetic samples are generated to augment the training dataset. The sample's generation, as well as the classification, is done based on 22 statistical features extracted from network flows. Truong-Huu et al. [5] studied the capability of a GAN for unsupervised network anomaly detection where 39 hand-designed features extracted from traffic flows and sessions are used as input. Results show that their proposed approach managed to obtain better results when compared to the autoencoder without any enhanced adversarial training. Doriguzzi-Corin et al. [15] proposed a spatial representation that enables a convolutional neural network to learn the correlation between 11 packet's features to detect Distributed Denial of Service (DDoS) traffic.

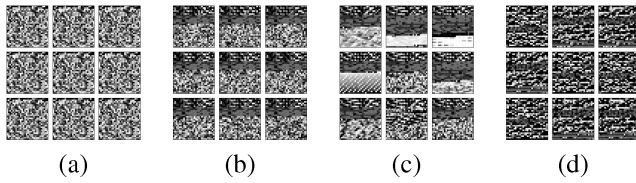


Fig. 1. Visualization of network flows from four distinct traffic classes of the USTC-TFC dataset. In this instance, 784 initial bytes of nine network flows (of four traffic classes) were reshaped into 28×28 grayscale images. (a) FTP. (b) Geodo. (c) Htbot. (d) World of Warcraft.

Network traffic feature learning is predominantly performed through ANN architectures like 1D-CNN, 2D-CNN, and Long Short-Term Memory (LSTM). Extracted bytes from network traffic flows (or packets) are kept sequential for the 1D-CNN and LSTM case, whereas for the 2D-CNNs, extracted bytes are seen as pixels of grayscale images, as illustrated in Figure 1. Wang et al. [16] proposed an approach that relies on the advantages of both 2D-CNNs and LSTMs to extract spatial-temporal features of network traffic. Results show that accuracy is improved when both architectures are combined. Wang et al. [17] proposed a supervised DL approach for malware traffic classification that uses 2D-CNNs to extract spatial features from headers and payloads of network flows and sessions. Two different choices of raw traffic images (named “ALL” and “L7”) dependent on the protocol layers considered to extract the input data are used to feed the classifier, showing that sessions with “ALL” are the most informative and reach elevate performance for all the metrics considered. Yu et al. [18] proposed a self-supervised learning 2D-CNN Stacked Autoencoder (SAE) for feature extraction, which is evaluated through different classification tasks with malware traffic data. Wang et al. [19] have shown that 1D-CNN outperforms 2D-CNN for encrypted traffic classification. Aceto et al. [20] performed an in-depth comparison on the application of Multilayer Perceptron (MLP), 1D-CNN, 2D-CNN, and LSTM architectures for encrypted mobile traffic classification. Numerical results indicated that 1D-CNN is a more appropriate choice for network traffic classification since it can better capture spatial dependencies between adjacent bytes in the network packets due to the nature of the input data that is, by definition, one-dimensional. Lotfollahi et al. [21] used 1D-CNN to automatically extract network traffic features and identify encrypted traffic to distinguish Virtual Private Network (VPN) and non-VPN traffic. Ahmad et al. [22] employed 1D-CNN-based classifier for early detection of network attacks. It is shown that a high degree of accuracy can be achieved by analyzing 1 to 3 packets.

The works mentioned above perform the task of traffic classification or anomaly detection based on labeled datasets. Recently, [6] proposed an “unsupervised” approach for anomaly detection, so-called D-PACK, in which only normal traffic is used during training. The model architecture is composed of 1D-CNN that performs feature extraction, followed by MLP softmax classifier given a labeled dataset of normal traffic, i.e., they assume the normal traffic is labeled into multiple classes (that is the reason why its respective

UD bullet is partially filled in Table I). The extracted features from an intermediate layer of the MLP are used as the input for a MLP-based AE. The anomaly score is based on a ℓ_2 -distance between the extracted features and the AE reconstruction. Results indicate that normal and malware traffic, such as the Mirai Botnet, can be effectively separated even when only two packets are used for detection. We implemented and included D-PACK in our experiments as a baseline model.

IV. METHODOLOGY

In this section, we present our so-called “ARCADE” proposed approach. The network flow preprocessing procedure is presented in Section IV-A. The model’s architecture is presented in Section IV-B. The AE distance metrics and adversarial training are presented in Section IV-C and Section IV-D, respectively. Finally, the anomaly score calculation is presented in Section IV-E.

A. Network Traffic Flow Preprocessing

Network traffic classification or anomaly detection can be performed at different granularity units, e.g., packet, flow, and session. It is worth noting that most of the works shown in Table I considered either flows or sessions as the relevant classification objects. A *network flow* is a unidirectional sequence of packets with the same 5-tuple (source IP, source port, destination IP, destination port, and transport-level protocol) exchanged between two endpoints. A *session* is defined as a bidirectional flow, including both directions of traffic. We increment the aforementioned flow definition by considering that a network flow is to be terminated or inactivated when the flow has not received a new packet within a specific flow timeout (e.g., 120 seconds). When the underlying network protocol is TCP, we consider the network connection closed (and the corresponding flow completed) upon detecting the first flow packet containing a FIN flag. Note that, in the case of TCP sessions, a network connection is considered closed when both sides have sent a FIN packet. Upon the termination of a network flow, unprocessed buffered packets should be discarded.

It is well known that the initial packets of each network flow contain the most information that allows for the discrimination between normal and abnormal activities [6], [20], [22], depicting the fundamental concept behind early detection approaches, which conduct the detection given a small number of initial packets of a flow. The smaller the number of packets required as input for the anomaly detection procedure, the lower the reaction time and overhead imposed by the DL method. Instead of analyzing every packet of a network flow on a time window, we use the n initial packets of a network flow as input. In this sense, n denotes the exact number of initial packets of a network flow required to form the input for ARCADE. For each active flow, n packets are buffered and trimmed into a fixed length of 100 bytes, starting with the header fields, i.e., packets are truncated to 100 bytes if larger, otherwise, padded with zeros. Packets are cleaned such that MAC and IP addresses are anonymized. Finally, bytes are normalized in $[0, 1]$ and packets concatenated into the

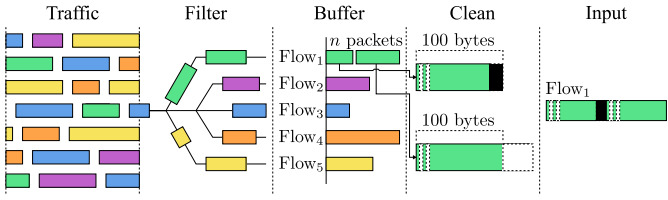


Fig. 2. An illustration of the proposed network traffic preprocessing pipeline with $n = 2$. Packets with the same color represent network flows. Traffic can be originated from a real-time packet sniffing or a .pcap file. Packets are filtered according to their 5-tuple, and n initial packets are buffered. MAC and IP addresses are masked, and according to their length, packets are truncated (if larger than 1) or padded with zeros (if smaller than 1). Finally, bytes are normalized, and packets are concatenated. Traffic preemption is not required.

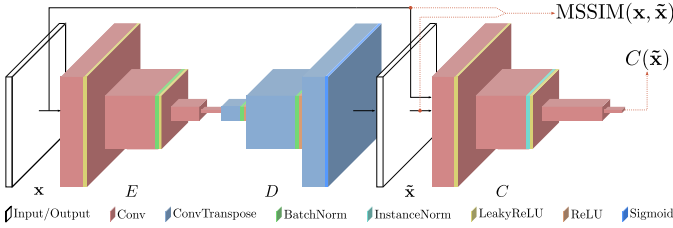


Fig. 3. An illustration of the proposed model architecture. Note that ARCADE is parametrized by 1D CNNs, as described in Section IV-B.

final input form, i.e., a sample $\mathbf{x}$ can be denoted as $\mathbf{x} \in \mathbb{R}^w$ where $w = 100n$ is the sequence length. Figure 2 illustrates the essential steps of the proposed network traffic flow preprocessing pipeline. Note that traffic preemption is not required. However, it is crucial to consider the reaction time, which relates to the processing power capabilities of the device in which the ARCADE will run. The analysis of the complexity and detection speed of ARCADE given different devices is provided in Section V-E.

B. Model Architecture

Several recent papers focus on improving training stability and the resulting quality of GAN samples [10], [12], [13]. Our proposed model is inspired by DCGAN [10], who introduce a convolutional generator network by removing fully connected layers and using convolutional layers and batch-normalization throughout the network. Strided convolutions replace pooling layers. This results in a more robust model with higher sample quality while reducing the model size and number of parameters to learn. Our proposed architecture shown in Figure 3 consists of two main components: (i) the AE (which can be seen as the generator) composed of an encoder E and a decoder D , and (ii) the critic C . Given the findings in [20], our functions E , D and C are parameterized by 1D-CNNs. Note that ARCADE can be easily modified to be used as an anomaly detection method for other anomaly detection tasks, such as image or time series anomaly detection. Moreover, the proposed adversarial regularization strategy can be applied to any AE, independent of its ANN architecture.

The AE consists of an encoder function $E : \mathbb{R}^w \mapsto \mathbb{R}^d$ and a decoder function $D : \mathbb{R}^d \mapsto \mathbb{R}^w$, where d denotes the dimensionality of the latent space. The overall encoding and

decoding process can be summarized as

$$\tilde{\mathbf{x}} = D(E(\mathbf{x})) = G(\mathbf{x}), \quad (5)$$

where $\tilde{\mathbf{x}}$ is the reconstruction of the input. The encoder uses strided convolutions to down-sample the input, followed by batch normalization and Leaky Rectified Linear Unit (Leaky ReLU). Differently from a deterministic pooling operation, strided convolutions allow the model to learn its own downsampling/upsampling strategy. The decoder uses strided transpose convolutions to up-sample the latent space, followed by Rectified Linear Unit (ReLU) and batch normalization. The critic function $C : \mathbb{R}^w \mapsto \mathbb{R}$, whose objective is to provide a score to the input $\mathbf{x}$ and the reconstruction $\tilde{\mathbf{x}}$, has a similar architecture to the encoder E . It also uses strided convolutions to down-sample the input and Leaky ReLU; however, following [13], we use layer normalization instead of batch normalization. The number of layers and filter size were defined so that ARCADE could effectively detect anomalies and still provide quick reaction time. Table X precisely presents the proposed model architecture.

C. Autoencoder Distance Metric

The core idea behind ARCADE is that the model must learn the normal traffic distribution to reconstruct it correctly. The hypothesis is that the model is conversely expected to fail to reconstruct attacks and malware traffic as it is never trained on such abnormal situations. A loss function must be defined to train an AE to reconstruct its input. For simplicity, a per-value $\mathcal{L}_2$ loss is typically used between the input $\mathbf{x}$ and reconstruction $\tilde{\mathbf{x}}$, and can be expressed as

$$\mathcal{L}_2(\mathbf{x}, \tilde{\mathbf{x}}) = \sum_{i=1}^w (\mathbf{x}_i - \tilde{\mathbf{x}}_i)^2, \quad (6)$$

where $\mathbf{x}_i$ is the i -th value in the sequence. During the evaluation phase, the per-value ℓ_2 -distance of $\mathbf{x}$ and $\tilde{\mathbf{x}}$ is computed to obtain the residual map.

As demonstrated by [23], AEs that make use of $\mathcal{L}_2$ loss may fail in some scenarios to detect structural differences between the input and their reconstruction. Adapting the loss and evaluation functions to the SSIM [24] that capture local inter-dependencies between the input and reconstruction regions can improve the AE's anomaly detection capabilities. This is also verified in this work, as demonstrated in Section V. The SSIM index defines a distance measure between two $K \times K$ patches $\mathbf{p}$ and $\mathbf{q}$ is given by

$$\text{SSIM}(\mathbf{p}, \mathbf{q}) = \frac{(2\mu_{\mathbf{p}}\mu_{\mathbf{q}} + c_1)(2\sigma_{\mathbf{p}\mathbf{q}} + c_2)}{(\mu_{\mathbf{p}}^2 + \mu_{\mathbf{q}}^2 + c_1)(\sigma_{\mathbf{p}}^2 + \sigma_{\mathbf{q}}^2 + c_2)}, \quad (7)$$

where $\mu_{\mathbf{p}}$ and $\mu_{\mathbf{q}}$ are the patches' mean intensity, $\sigma_{\mathbf{p}}^2$ and $\sigma_{\mathbf{q}}^2$ are the variances, and $\sigma_{\mathbf{p}\mathbf{q}}$ the covariance. The constants c_1 and c_2 ensure numerical stability and are typically set to $c_1 = 0.01$ and $c_2 = 0.03$.

The SSIM is commonly used to measure the similarity between images, performed by sliding a $K \times K$ window that moves pixel-by-pixel. Since in our case $\mathbf{x}$ is a sequence, we split it into n subsequences of length l , i.e., each subsequence

$$\mathbf{x}'_i = (\mathbf{x}_j \in [0, 1] : j \in \{1 + (i-1)l, \dots, il\}),$$

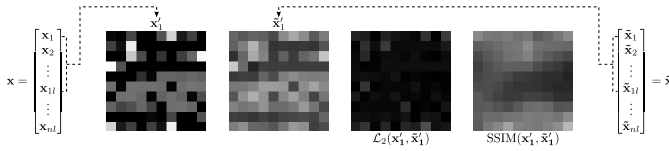


Fig. 4. An illustration of the advantages of the SSIM over $\mathcal{L}_2$ for the segmentation of the discrepancies between a subset of bytes of a packet and their respective reconstructions.

where $i \in \{1, 2, \dots, n\}$ and $l = 100$ can be seen as the subset of 100 bytes of the i -th packet that was originally used to compose the sequence $\mathbf{x}$. Finally, subsequences are reshaped $\mathbf{x}'_i \in \mathbb{R}^l \mapsto \mathbf{x}'_i \in \mathbb{R}^{K \times K}$, where $K = \sqrt{l}$ and l is a perfect square number. An illustration of this procedure is shown in Figure 4. The mean SSIM gives the overall structural similarity measure of the sequence (MSSIM), defined as

$$\text{MSSIM}(\mathbf{x}, \tilde{\mathbf{x}}) = \frac{1}{nM} \sum_{i=1}^n \sum_{j=1}^M \text{SSIM}(\mathbf{x}'_i(j), \tilde{\mathbf{x}}'_i(j)), \quad (8)$$

where M is the number of local windows, and $\mathbf{x}'_i(j)$ and $\tilde{\mathbf{x}}'_i(j)$ are the contents at the j -th local window of the i -th subsequences $\mathbf{x}'_i$ and $\tilde{\mathbf{x}}'_i$.

D. Adversarial Training

We address the generalization problem by regularizing the AE through adversarial training. Additionally to maximizing MSSIM, we further maximize the reconstruction scores provided by the critic C . By doing so, besides generating contextually similar reconstructions, the AE must reconstruct normal-like samples as faithfully as possible so the scores given by the critic C are maximized. During training, the AE is optimized to maximize

$$\mathcal{L}_G = \mathbb{E}_{\mathbf{x} \sim \mathbb{P}_r} [\text{MSSIM}(\mathbf{x}, \tilde{\mathbf{x}}) + \lambda_G C(\tilde{\mathbf{x}})], \quad (9)$$

where λ_G is the regularization coefficient that balance the terms of the AE's objective function.

In Equation (9), it is assumed that critic C can provide high scores for real normal traffic samples and low scores for reconstruction. To do so, the critic C must learn the normal and reconstruction data distributions. Therefore, during training, the critic C is optimized to maximize

$$\mathcal{L}_C = \mathbb{E}_{\mathbf{x} \sim \mathbb{P}_r} [C(\mathbf{x}) - C(\tilde{\mathbf{x}})] + \lambda_C \mathcal{L}_{GP}, \quad (10)$$

where $\mathcal{L}_{GP}$ is given by Equation (4), and $\lambda_C = 10$ as suggested in [13]. Our adversarial training strategy is based on the WGAN-GP framework described in Section II-B. Algorithm 1 summarizes the essential steps of the proposed adversarial training.

E. Anomaly Score

An anomaly score $\mathcal{A}(\mathbf{x})$ is a function that provides a score to a sample $\mathbf{x}$ in the test set concerning samples in the training set. Samples with more significant anomaly scores are considered more likely to be anomalous. Traditionally, AE strategies for anomaly detection rely on the reconstruction error between the

Algorithm 1 Proposed Adversarial Training. We Use $m = 64$, $\lambda_C = 10$, $\lambda_G = 100$, $\alpha = 1e-4$, $\beta_1 = 0$, and $\beta_2 = 0.9$

Require: Batch size m , maximum training iterations $\text{max}_{\text{epoch}}$, C penalty coefficients λ_C and λ_G , Adam hyperparameters α, β_1, β_2 , critic and autoencoder initial parameters ψ_0 and θ_0 , respectively.

```

1: while current epoch is smaller than  $\text{max}_{\text{epoch}}$  do
2:   Sample a batch of normal traffic samples  $\{\mathbf{x}^{(i)}\}_{i=1}^m \sim \mathbb{P}_r$ 
3:    $\tilde{\mathbf{x}} \leftarrow G_\theta(\mathbf{x})$ 
4:   for  $i \leftarrow 1$  to  $m$  do
5:     Sample a random number  $\epsilon \sim U(0, 1)$ 
6:      $\hat{\mathbf{x}} \leftarrow \epsilon \mathbf{x}^{(i)} + (1 - \epsilon) \tilde{\mathbf{x}}^{(i)}$ 
7:      $\mathcal{L}_C^{(i)} \leftarrow C_\psi(\mathbf{x}^{(i)}) - C_\psi(\tilde{\mathbf{x}}^{(i)}) + \lambda_C (\|\nabla_{\tilde{\mathbf{x}}} C_\psi(\hat{\mathbf{x}})\|_2 - 1)^2$ 
8:    $\psi \leftarrow \text{Adam}(\nabla_\psi \frac{1}{m} \sum_{i=1}^m \mathcal{L}_C^{(i)}, \psi, \alpha, \beta_1, \beta_2)$ 
9:    $\mathcal{L}_G \leftarrow \text{MSSIM}(\mathbf{x}, \tilde{\mathbf{x}}) + \lambda_G C_\psi(\tilde{\mathbf{x}})$ 
10:   $\theta \leftarrow \text{Adam}(\nabla_\theta \frac{1}{m} \sum_{i=1}^m -\mathcal{L}_G^{(i)}, \theta, \alpha, \beta_1, \beta_2)$ 

```

input, and the reconstruction of the input has been used as an anomaly score. Another widely adopted anomaly score is the feature matching error based on an intermediate layer of the discriminator [5], [26].

Experiments with the feature matching error as an anomaly score did not significantly improve ARCADE's performance. At the same time, it considerably increased the inference time since it is required to feed $\mathbf{x}$ and $\tilde{\mathbf{x}}$ through C for feature extraction. Similarly, we found that using MSSIM as an anomaly score leads to a more discriminative anomaly function when compared to $\mathcal{L}_2$. However, the gains in efficiency are not meaningful enough to justify the loss in efficiency due to the SSIM's complexity. Therefore, for a given sample $\mathbf{x}$ in the test set, its anomaly score computed using ARCADE is denoted as $\mathcal{A}(\mathbf{x}) = \mathcal{L}_2(\mathbf{x}, \tilde{\mathbf{x}})$.

V. EXPERIMENTAL EVALUATION

The present section investigates and compares the performance of ARCADE with baselines on three network traffic datasets. The considered datasets and baselines are described in Section V-A and Section V-B, respectively. Implementation, training details, and hyper-parameter tuning are described in Section V-C. In Section V-D, we assess the effectiveness of ARCADE and baselines on the three considered datasets. Section V-E present the analysis of the complexity and the detection speed of ARCADE and D-PACK baseline.

A. Datasets Description

We used three datasets to evaluate the performance of the proposed approach with real-world normal and malicious network traffic: ISCX-IDS [33], USTC-TFC [17], and MIRAI-RGU [34]. The choice of datasets is based on the requirement for raw network traffic. The selected datasets are among the most well-known datasets for intrusion detection, which provide raw network traffic (.pcap) in addition to hand-designed features (.csv). For example, the KDD'99 and NSL-KDD datasets provide only hand-designed extracted features, which limits their use in this work. It is worth noting that the number of flows presented in dataset Table II, III, and IV described

TABLE IV
MIRAI-RGU DATASET

Normal Traffic Type	# of Flows
HTTP	3,526,212
Anomaly Traffic Type	# of Flows
Infection	2,795,422
GREETH Flood	67,116
VSE Flood	4,990
ACK Flood	137,838
DNS Flood	9,704
HTTP Flood	272
UDP PLAIN Flood	18
UDP Flood	32,418
SYN Flood	47,682
GREIP Flood	77,293

below are the number of flows achieved after the preprocessing procedure proposed in Section IV-A.

The ISCX-IDS dataset [33] is a realistic-like dataset originally proposed for the development of enhanced intrusion detection and anomaly-based approaches. The network traffic was collected for seven days. Packets collected on the first and sixth days are normal traffic. Normal and attack packets are collected on the second and third days. In the fourth, fifth, and seventh days, besides the normal traffic, HTTP DoS, DDoS using an IRC Botnet, and Brute Force (BF) SSH packets are collected, respectively. Table II provides an overview of the ISCX-IDS dataset. The USTC-TFC dataset [17] includes ten classes of normal traffic and ten classes of malware traffic from public websites, which were collected from a real network environment from 2011 to 2015. Table III provides an overview of the USTC-TFC dataset. The MIRAI-RGU dataset includes normal traffic from Internet of the Things (IoT) Internet Protocol (IP) cameras and ten classes of malicious traffic from the Mirai botnet malware, such as HTTP flood, UDP flood, DNS flood, Mirai infection traffic, VSE flood, GREIP flood, GREETH flood, TCP ACK flood, TCP SYN flood, and UDPPLAIN flood. Table IV provides an overview of the MIRAI-RGU dataset.

We split each dataset into training, validation, and test sets. The training set is composed only of normal samples. Normal and anomaly samples are used only for testing and validation. We balance the test set such that each subset of classes in the test set presents the same number of samples. Note that the normal traffic in the test set is not a subset of the training set. The validation set is composed of 5% of the samples of each class from the test set, randomly selected and removed for validation purposes.

B. Competing Methods

We compare ARCADE to three shallow and four deep learning methods for anomaly detection. The chosen shallow baselines are well-known methods typically applied to anomaly detection problems and commonly used as a benchmark for novel anomaly detection methods. D-PACK is the state-of-the-art unsupervised network anomaly detection method using raw network traffic bytes as input. We implemented and used D-PACK's effectiveness and efficiency as a baseline for

ARCADE. GANomaly was chosen due to the similarities in the adversarial regularization strategies. Comparing ARCADE with GANomaly allows us to assess the effectiveness of our proposed adversarial strategy. Moreover, comparing ARCADE with AE- ℓ_2 and AE-SSIM allows us to confirm the findings presented by [23] and also assess the effectiveness of the proposed adversarial regularization since ARCADE with $\lambda_G = 0$ is equivalent to AE-SSIM. We also implemented and performed experiments with probabilistic models such as Variational Autoencoder (VAE) and Adversarial Autoencoder (AAE); however, they did not produce satisfactory results when compared to deterministic AEs. Therefore, their results are not reported. Below we describe each competing method and its respective parameters.

1) *Shallow Baselines*: (i) One-Class SVM (OC-SVM) [35] with Gaussian kernel. We optimize the hyperparameters γ and ν via grid search using the validation set with $\gamma \in \{2^{-10}, 2^{-9}, \dots, 2^0\}$, and $\nu \in \{0.01, 0.02, \dots, 0.1\}$. (ii) Kernel density estimation (KDE). We optimize the bandwidth h of the Gaussian kernel via grid search, given ten values spaced evenly between -1 to 1 on a logarithmic scale. (iii) Isolation Forest (IF) [36]. As recommended in the original work, we set the number of trees to 100 and the subsampling size to 256. For all three shallow baselines, we reduce the dimensionality of the data via Principal Component Analysis (PCA), where we choose the minimum number of eigenvectors such that at least 95% of the variance is retained.

2) *Deep Baselines*: (i) D-PACK [6], recently proposed for unsupervised network anomaly detection, can be considered the state-of-the-art DL method for the task. D-PACK's performance serves as a point of comparison for ARCADE's effectiveness and efficiency. The original D-PACK formulation assumes that normal traffic is split into multiple classes, as in the USTC-TFC dataset. However, this is not the circumstance for most public datasets, such as the other two datasets considered here. We empirically assessed that removing the softmax classifier degrades the method's efficiency. Therefore, we keep the original D-PACK formulation even for datasets without labeled normal training data. The network architecture, training strategy, and hyperparameters were kept as recommended in the original work. (ii) GANomaly [28] was originally proposed for image anomaly detection. Here, we do not employ it as an out-of-the-box anomaly detection approach. However, we use its adversary training framework with the proposed 1D-CNN model architecture presented in Section IV-B. The idea is to compare GANomaly's training strategy with our proposed adversarial training strategy. Note that GANomaly defines the generator G as an encoder-decoder-encoder. Therefore, a second encoder E' with the same architecture of E (without sharing parameters) is added to the proposed AE, where the input of E' is the outcome of the decoder D , i.e., the input for encoder E' is the reconstruction of the input. Finally, we modify the critic C to align with their proposed discriminator D . We modify C , so batch normalization is used instead of layer normalization, and a Sigmoid activation function is added after the last layer. The anomaly score is given by the ℓ_2 -distance between the latent space of E and the latent space of E' . We performed grid search

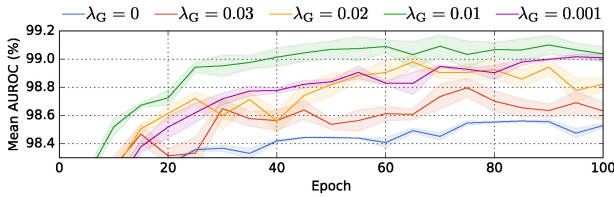


Fig. 5. ARCADE’s mean AUROC (%) convergence given varying values for the adversarial regularization coefficient λ_G . In the case where $\lambda_G = 0$, results are equivalent to the AE-SSIM.

TABLE V
ARCADE’S MEAN AUROC (%) ON THE THREE CONSIDERED DATASETS GIVEN VARYING INPUT SIZES. RESULTS ARE IN THE FORMAT *mean* $\pm$ *std*. OBTAINED OVER 10-FOLDS

Dataset	Number of Packets				
	2	3	4	5	6
ISCX-IDS	86.7 $\pm$ 0.1	96.9 $\pm$ 0.5	99.0 $\pm$ 0.1	99.3 $\pm$ 0.0	99.1 $\pm$ 0.0
USTC-TFC	99.9 $\pm$ 0.0	100 $\pm$ 0.0	100 $\pm$ 0.0	100 $\pm$ 0.0	100 $\pm$ 0.0
MIRAI-RGU	99.6 $\pm$ 0.0	99.9 $\pm$ 0.0	99.9 $\pm$ 0.0	99.9 $\pm$ 0.0	99.8 $\pm$ 0.0

TABLE VI
AUROC (%) OF ARCADE AND SHALLOW BASELINES. RESULTS ARE IN THE FORMAT *mean* $\pm$ *std*. OBTAINED OVER 10-FOLDS. WE PRESENT RESULTS FOR THE ISCX-IDS WITH $n \in \{2, 5\}$, DENOTED AS ISCX-IDSⁿ

Dataset	Shallow Baselines			Proposed
	OC-SVM	KDE	IF	ARCADE
ISCX-IDS ²	79.03 $\pm$ 0.0	80.46 $\pm$ 0.0	74.32 $\pm$ 0.1	86.73$\pm$0.1
ISCX-IDS ⁵	87.24 $\pm$ 0.0	68.01 $\pm$ 0.0	78.44 $\pm$ 0.1	99.32$\pm$0.0
USTC-TFC	96.86 $\pm$ 0.0	89.91 $\pm$ 0.0	81.45 $\pm$ 0.1	99.99$\pm$0.0
MIRAI-RGU	98.02 $\pm$ 0.0	98.55 $\pm$ 0.0	95.15 $\pm$ 0.9	99.99$\pm$0.0

optimize $w_{\text{rec}} \in \{50, 75, 100, 125, 150\}$ and results suggest that $w_{\text{rec}} = 75$ lead to best results. All the other parameters were kept as suggested in the original work. (iii) AE- ℓ_2 is an AE with the same proposed network architecture in Section IV-B, where $\mathcal{L}_2$ loss is used as distance metric during training, and $\mathcal{L}_2$ is also used for the anomaly score computation. (iv) AE-SSIM is an AE with the same proposed network architecture in Section IV-B, where MSSIM loss is used for training, and $\mathcal{L}_2$ is used for computing the anomaly scores. In this work, we used the PyTorch Image Quality (PIQ) [37] implementation of the SSIM loss with Gaussian kernel and kernel size $K = 3$, obtained through a grid search optimization with $K \in \{3, 5, 7, 9\}$.

C. Training Recipe and Ablation Study

The training objective (described in Section IV-D) is optimized via Adam optimizer [38] with $\alpha = 1e-4$, $\beta_1 = 0$, and $\beta_2 = 0.9$. It is worth noting again that Algorithm 1 describes the main steps of the proposed adversarial training procedure. Additionally, we employ for all approaches a two-phase (“searching” and “fine-tuning”) learning rate $1e-4$ for 100 epochs. In the fine-tuning phase, we train with the learning rate $1e-5$ for another 50 epochs. The latent size d is computed with PCA, equivalent to the minimum number of

eigenvectors such that the sum of their explained variance is at least 95%, i.e., $d \approx 50$ with $n = 2$ for all three datasets.

Given the hyperparameters and training recipe above, we performed ablation experiments to assess the performance of ARCADE with and without the proposed adversarial regularization and varying input sizes. The validation set was used for the ablation experiments. To assess the effectiveness of the proposed adversarial regularization, we performed experiments with the ISCX-IDS dataset with $n = 5$, and $\lambda_G \in \{0, 0.001, 0.01, 0.02, 0.03\}$. Figure 5 illustrates the mean AUROC convergence (lines) and standard deviation (error bars amplified 50 times for visualization purposes). We can verify that the proposed adversarial-based regularization improves the capabilities of the AE for network anomaly detection concerning the same AE without the adversarial regularization, i.e., with $\lambda_G = 0$ ARCADE is equivalent to AE-SSIM. The proposed adversarial training strategy can be exploited to improve the network anomaly detection capabilities of similar DL approaches, especially for scenarios where increasing the model size is not an option due to hardware constraints. Based on these results, we fix the adversarial regularization coefficient to $\lambda_G = 0.01$ for all the following experiments. We also analyze the ARCADE performance given different input sizes. Table V presents the mean AUROC and standard deviations on the three datasets with $n \in \{2, 3, 4, 5, 6\}$. ARCADE achieves nearly 100 AUROC with $n = 2$ on the USTC-TFC and MIRAI-RGU datasets. For the ISCX-IDS dataset, the method achieves 86.7 and 99.1 AUROC with 2 and 4 packets, respectively. The following experiments further investigate the considerable difference in performance given varying values of n . For the MIRAI-RGU dataset, the AUROC decreases with $n > 5$. Scaling the model depth and width given the input size could help since, for larger input sizes, more layers and channels would lead to an increased receptive field and more fine-grained patterns. Note that additional ablation experiments concerning the ℓ_2 and SSIM loss functions are provided in the following section.

D. Network Traffic Anomaly Detection Results

We now systematically compare the proposed ARCADE’s effectiveness with the baselines. Table VI presents the results of the considered shallow baselines on the three network traffic datasets. ARCADE outperforms all of its shallow competitors. Table VII presents the results of ARCADE and considered deep baselines. Here, we expand the evaluations to include a one-class anomaly detection setting, where each anomaly class is evaluated separately. Therefore, the table also includes the AUROC and F1-score concerning the evaluation performed exclusively on each anomaly class presented in each dataset. Note that the anomaly samples used for this evaluation are not necessarily a subset of the test set and were fixed for all methods. This allows each method to be evaluated separately against each attack or malware in each dataset.

The results for the deep baselines, considering normal and all anomalies, show that ARCADE outperforms all other methods on the three considered datasets. The methods rank ARCADE, AE-SSIM, AE- ℓ_2 , GANomaly, and D-PACK for

TABLE VII
AUROC AND F1-SCORE (%) OF ARCADE AND DEEP BASELINES. EACH METHOD WAS TRAINED EXCLUSIVELY ON NORMAL NETWORK TRAFFIC, AND THE RESULTS ARE IN THE FORMAT MEAN ($\pm$ STD.) OBTAINED OVER 10-FOLDS.
FOR THE ISCX-IDS, WE RUN TWO EXPERIMENTS WITH $n \in \{2, 5\}$

ISCX-IDS	D-PACK		AE- ℓ_2		AE-SSIM		GANomaly		ARCADE	
	AUROC	F1	AUROC	F1	AUROC	F1	AUROC	F1	AUROC	F1
Input Size 200										
Infiltration	99.36 (± 0.1)	99.07	99.20 (± 0.1)	99.04	99.13 (± 0.0)	99.10	99.12 (± 0.0)	98.98	99.43 (± 0.0)	99.16
HTTP DoS	83.63 (± 2.8)	67.53	77.59 (± 0.3)	61.96	77.98 (± 0.4)	62.15	79.63 (± 0.5)	66.06	81.22 (± 2.2)	68.70
DDoS	47.06 (± 5.2)	44.53	43.31 (± 0.5)	46.51	44.05 (± 1.4)	44.51	40.97 (± 1.2)	21.98	55.35 (± 1.0)	66.61
BF SSH	97.35 (± 1.0)	86.51	99.66 (± 0.0)	98.73	99.68 (± 0.0)	98.80	99.29 (± 0.1)	98.20	99.76 (± 0.1)	99.24
All anomalies	80.86 (± 1.8)	72.31	83.05 (± 0.4)	75.19	83.63 (± 0.3)	75.35	82.66 (± 0.3)	75.98	86.73 (± 0.1)	77.19
Input Size 500										
Infiltration	96.68 (± 2.4)	93.88	99.32 (± 0.0)	99.01	99.35 (± 0.0)	99.09	99.26 (± 0.0)	98.69	99.62 (± 0.0)	99.17
HTTP DoS	93.56 (± 2.0)	88.99	92.48 (± 0.0)	90.84	92.28 (± 0.1)	90.67	92.59 (± 0.3)	91.56	93.72 (± 0.4)	91.95
DDoS	95.40 (± 1.2)	94.73	89.89 (± 0.2)	92.78	90.26 (± 0.2)	92.59	89.55 (± 0.6)	92.46	91.04 (± 0.1)	93.19
BF SSH	99.11 (± 0.3)	96.91	99.81 (± 0.0)	99.47	99.91 (± 0.0)	99.49	99.99 (± 0.0)	99.66	99.96 (± 0.0)	99.63
All anomalies	96.38 (± 2.5)	93.14	98.58 (± 0.0)	96.72	98.62 (± 0.0)	96.90	98.58 (± 0.0)	97.22	99.32 (± 0.0)	97.29
USTC-TFC	D-PACK		AE- ℓ_2		AE-SSIM		GANomaly		ARCADE	
	AUROC	F1	AUROC	F1	AUROC	F1	AUROC	F1	AUROC	F1
Cridex	99.29 (± 0.1)	94.05	100 (± 0.0)	100	100 (± 0.0)	100	99.91 (± 0.1)	98.88	100 (± 0.0)	100
Geodo	99.28 (± 0.2)	94.08	99.99 (± 0.0)	99.91	99.99 (± 0.0)	99.88	99.77 (± 0.0)	96.35	100 (± 0.0)	100
Htbot	99.47 (± 0.0)	94.41	100 (± 0.0)	100	100 (± 0.0)	100	99.97 (± 0.0)	98.55	100 (± 0.0)	100
Miuref	99.42 (± 0.1)	94.63	100 (± 0.0)	99.97	99.99 (± 0.0)	99.86	99.83 (± 0.0)	97.73	100 (± 0.0)	100
Neris	99.76 (± 0.0)	95.05	99.99 (± 0.0)	99.83	100 (± 0.0)	99.97	99.98 (± 0.0)	99.37	100 (± 0.0)	100
Nsis-ay	99.72 (± 0.1)	93.89	99.99 (± 0.0)	99.94	100 (± 0.0)	100	99.99 (± 0.0)	99.41	100 (± 0.0)	100
Shifu	99.51 (± 0.1)	95.63	100 (± 0.0)	100	100 (± 0.0)	100	99.96 (± 0.0)	98.68	100 (± 0.0)	100
Tinba	99.92 (± 0.0)	96.12	99.99 (± 0.0)	99.97	100 (± 0.0)	100	99.99 (± 0.0)	99.91	100 (± 0.0)	100
Virut	99.80 (± 0.1)	95.96	99.99 (± 0.0)	99.91	100 (± 0.0)	100	99.99 (± 0.0)	99.45	100 (± 0.0)	100
Zeus	99.01 (± 0.2)	88.52	100 (± 0.0)	100	100 (± 0.0)	100	99.90 (± 0.0)	98.16	100 (± 0.0)	100
All anomalies	99.59 (± 0.2)	98.77	99.99 (± 0.0)	99.89	99.99 (± 0.0)	99.93	99.81 (± 0.0)	99.40	99.99 (± 0.0)	99.98
MIRAI-RGU	D-PACK		AE- ℓ_2		AE-SSIM		GANomaly		ARCADE	
	AUROC	F1	AUROC	F1	AUROC	F1	AUROC	F1	AUROC	F1
Infection	99.66 (± 0.1)	98.33	99.74 (± 0.0)	99.68	99.77 (± 0.0)	99.83	99.80 (± 0.0)	99.47	99.99 (± 0.0)	99.83
GREETH Flood	99.77 (± 0.1)	99.47	99.96 (± 0.0)	99.85	99.98 (± 0.0)	99.88	99.97 (± 0.0)	99.88	99.97 (± 0.0)	99.86
VSE Flood	99.70 (± 0.2)	99.38	99.99 (± 0.0)	99.85	99.99 (± 0.0)	99.85	99.96 (± 0.0)	99.80	99.99 (± 0.0)	99.86
ACK Flood	99.90 (± 0.0)	99.41	99.99 (± 0.0)	99.81	99.99 (± 0.0)	99.89	99.99 (± 0.0)	99.89	100 (± 0.0)	99.89
DNS Flood	99.82 (± 0.1)	99.48	99.99 (± 0.0)	99.78	99.99 (± 0.0)	99.85	99.98 (± 0.0)	99.86	99.99 (± 0.0)	99.92
HTTP Flood	100 (± 0.0)	99.73	100 (± 0.0)	99.80	100 (± 0.0)	99.80	100 (± 0.0)	99.80	100 (± 0.0)	99.80
UDP Plain Flood	100 (± 0.0)	97.29	100 (± 0.0)	97.29	100 (± 0.0)	97.29	100 (± 0.0)	97.29	100 (± 0.0)	97.29
UDP Flood	99.69 (± 0.2)	99.33	99.96 (± 0.0)	99.86	99.94 (± 0.0)	99.83	99.99 (± 0.0)	99.85	99.96 (± 0.0)	99.84
SYN Flood	99.90 (± 0.0)	99.69	99.99 (± 0.0)	99.81	100 (± 0.0)	99.94	99.99 (± 0.0)	99.88	100 (± 0.0)	99.96
GREIP Flood	99.77 (± 0.1)	99.57	100 (± 0.0)	99.96	100 (± 0.0)	99.96	100 (± 0.0)	99.96	100 (± 0.0)	99.96
All anomalies	99.72 (± 0.1)	99.81	99.92 (± 0.0)	99.96	99.93 (± 0.0)	99.97	99.92 (± 0.0)	99.94	99.99 (± 0.0)	99.97

results on the ISCX-IDS with $n = 2$, USTC-TFC with $n = 2$, and MIRAI-RGU with $n = 2$. In experiments with the ISCX-IDS with $n = 5$, the methods rank ARCADE, GANomaly, AE-SSIM, AE- ℓ_2 , and D-PACK. Despite having approximately 20 times more parameters than the proposed model, D-PACK achieved the worst results among the deep baselines. Results for the AE-SSIM and AE- ℓ_2 , similarly to the results provided in [23], show that using SSIM as a distance metric during training can improve the AE's capabilities in detecting anomalies. ARCADE, which also uses SSIM as a distance metric during training and employs the proposed adversarial regularization strategy, achieved better results than AE-SSIM, emphasizing the advantages of the proposed adversarial training strategy. The GANomaly framework, comprised of its distinct model architecture, adversarial training strategy, and anomaly score, did not achieve better results than ARCADE. It

is worth noting that GANomaly used the same AE architecture as ARCADE with the requirement of an additional encoder, as described in Section V-B2.

The isolated validations for the ISCX-IDS with $n = 2$ show that ARCADE achieved the best F1-score values for all classes and best AUROC values for Infiltration, DDoS, and BF SSH, where D-PACK achieved the best AUROC for HTTP DoS. With $n = 5$, ARCADE achieved the best results for Infiltration and HTTP DoS, where D-PACK achieved the best results for DDoS, and GANomaly achieved the best results for BF SSH. The low performance of the considered methods on DoS and DDoS with $n = 2$ indicates that analyzing the spatial relation between bytes among multiple subsequent packets is essential to detect such attacks. A single packet of a flood attack does not characterize an anomaly; e.g., an SYN packet can be found within the normal traffic. However, multiple SYN

TABLE VIII
THE ACCURACY, PRECISION, RECALL, AND F1-SCORE VALUES IN % OF ARCADE AND D-PACK FOR THE 99TH PERCENTILE AND MAXIMUM THRESHOLDS. RESULTS ARE IN THE FORMAT OF THE MEAN ($\pm$ STD.) OBTAINED OVER TEN SEEDS

Dataset	D-PACK				ARCADE			
	Accuracy	Precision	Recall	F1-score	Accuracy	Precision	Recall	F1-score
99th percentile threshold								
ISCX-IDS	75.91 (± 0.08)	97.98 (± 0.01)	52.83 (± 0.15)	67.43 (± 0.13)	93.07 (± 0.01)	98.86 (± 0.00)	87.15 (± 0.01)	92.63 (± 0.01)
USTC-TFC	97.71 (± 0.00)	98.97 (± 0.00)	96.43 (± 0.01)	97.68 (± 0.00)	99.49 (± 0.00)	99.00 (± 0.00)	100 (± 0.00)	99.50 (± 0.00)
Mirai-RGU	98.50 (± 0.01)	99.87 (± 0.00)	98.44 (± 0.02)	99.14 (± 0.01)	99.89 (± 0.00)	99.87 (± 0.00)	100 (± 0.00)	99.93 (± 0.00)
Mean	90.70	98.94	82.56	87.08	97.48	99.24	95.71	97.35
Maximum threshold								
ISCX-IDS	50.00 (± 0.00)	100 (± 0.00)	0.016 (± 0.00)	0.033 (± 0.00)	60.11 (± 0.04)	100 (± 0.00)	20.21 (± 0.07)	33.01 (± 0.10)
USTC-TFC	50.83 (± 0.00)	100 (± 0.00)	1.627 (± 0.00)	3.202 (± 0.00)	99.88 (± 0.00)	100 (± 0.00)	99.77 (± 0.00)	99.88 (± 0.00)
Mirai-RGU	32.67 (± 0.11)	100 (± 0.00)	24.45 (± 0.12)	37.75 (± 0.16)	76.52 (± 0.02)	100 (± 0.00)	73.65 (± 0.02)	84.81 (± 0.01)
Mean	44.5	100	8.69	13.66	78.83	100	64.54	72.56

packets in sequence can be characterized as an SYN flood attack. This indicates that the spatial relation between bytes among subsequent packets is essential to detect such attacks. In isolated experiments with anomaly classes from the USTC-TFC dataset, ARCADE achieved maximum results with 100 AUROC and 100 F1-score in all malware classes. Results from the isolated experiments with anomaly classes from the MIRAI-RGU show that, if we consider D-PACK, AE- ℓ_2 , AE-SSIM, and GANomaly, there is no clear winner. ARCADE achieved the best AUROC and F1-score values on the 8 and 6 classes, respectively. GANomaly ranked second with four best AUROC and three best F1-score values.

In practice, a threshold value must be set to distinguish between normal and anomalous traffic based on the anomaly score distribution of the normal traffic. In a supervised scenario where the normal and known anomalies' anomaly score distribution does not overlap, the maximum anomaly score of the normal traffic can lead to 100% Detection Rate (DR) and 0% FAR. This is commonly adopted since it leads to small FAR. To avoid the impact of extreme maximum anomaly scores, the 99th percentile of the anomaly score distribution of the normal traffic can be used as an alternative. The downside of this approach is that approximately 1% FAR is expected. Regardless, the definition of the threshold is problem-dependent and is strongly related to IDS architecture altogether, e.g., in a hybrid IDS (anomaly-based and signature-based), where the anomaly-based method is used as a filter to avoid unnecessary signature verification, a high threshold could lead to low detection rates. In this case, a lower threshold, such as the 99th percentile (or even smaller), would be preferable since the signature-based approach would further validate false positives.

We further compare ARCADE and D-PACK considering accuracy, precision, recall, and F1-score given two thresholds: (i) the 99th percentile, and (ii) the maximum value of the normal traffic anomaly scores. This comparison aims to analyze the effectiveness of ARCADE compared to the D-PACK baseline, originally proposed for network anomaly detection. The other deep baselines use the same model architecture as ARCADE and can be seen as contributions to this work

we implemented. Table VIII presents the accuracy, precision, recall, and F1-score of ARCADE and D-PACK given both thresholds, with $n = 2$ for the USTC-TFC and MIRAI-RGU datasets, and $n = 5$ for the ISCX-IDS dataset. The results of the 99th threshold show that ARCADE achieved the highest recall rate for the USTC-TFC and MIRAI-RGU datasets. This is because ARCADE produced no false negatives. ARCADE achieved an 11.79% higher F1-score than D-PACK. When the maximum threshold is used, the ARCADE enhancement in performance is more clearly seen. As expected, both approaches were able to achieve the highest precision. However, D-PACK only achieved 8.69% mean recall, while ARCADE achieved 64.54%. This is an improvement of 642.69%. Figure 6 shows the anomaly score distribution of ARCADE and D-PACK computed using the model parameters that led to the best AUROC obtained over 10-folds on the three datasets. The detection rate is reported and calculated using the 99th percentile threshold, also presented in the figures. When considering the best model parameters and a 99th percentile threshold, ARCADE outperformed D-PACK in terms of detection rates by 22.35%, 3.44%, and 0.14% on the ISCX-IDS, USTC-TFC, and Mirai-RGU datasets, respectively.

E. Model Complexity and Detection Speed

Here we evaluate the efficiency of ARCADE by comparing with D-PACK the number of samples processed per second, model sizes, and floating-point operations (FLOPS). Figure 7 presents ARCADE and D-PACK efficiency, effectiveness, and model size on the ISCX-IDS with $n = 2$ and $n = 5$, where our ARCADE significantly outperforms D-PACK in all evaluated measures. We analyze the detection speed performance of ARCADE and D-PACK by assessing how many samples per second they can process in different environments with distinct processing capabilities that we categorize as edge, fog, and cloud. The device specifications and the experimental environment are summarized in Table XI. We consider a Raspberry Pi 4B as an edge device, UP Xtreme and Jetson Xavier NX as fog devices, and a desktop personal computer with an AMD Ryzen Threadripper 3970X 32-core CPU,

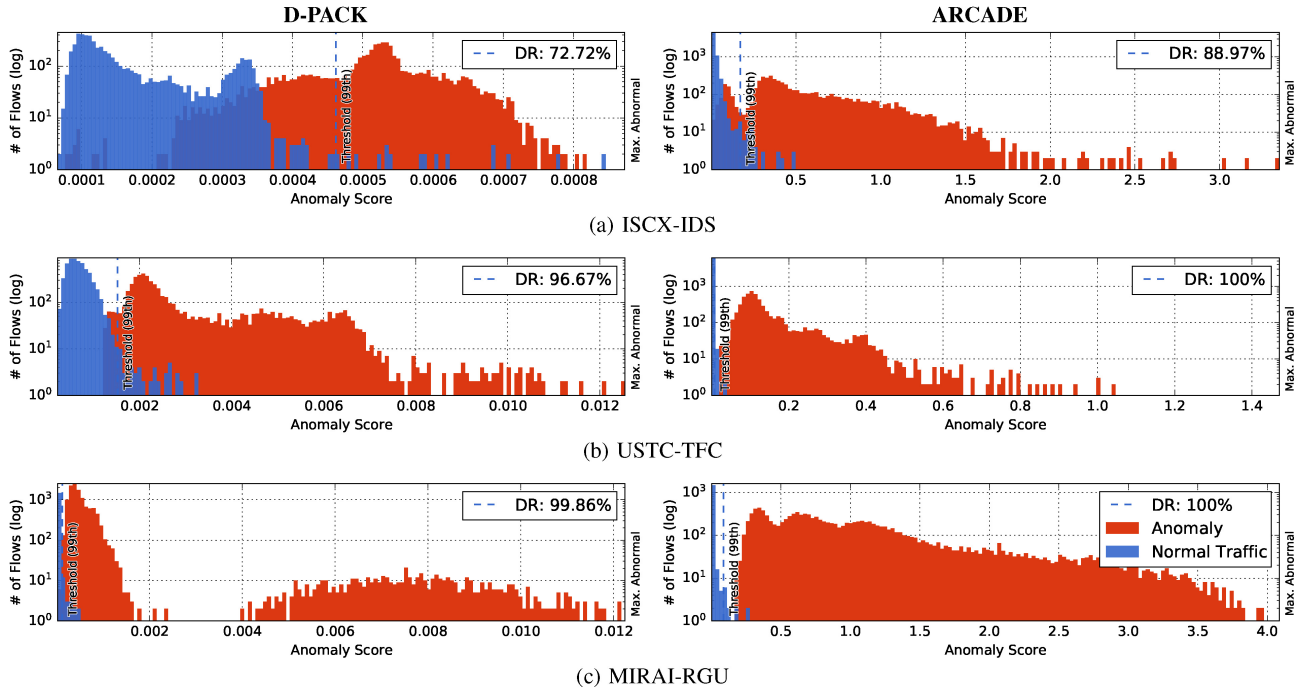


Fig. 6. The anomaly score distributions for normal and abnormal traffic from the test set of each considered dataset. Anomaly scores were computed using the best model's parameters over 10-folds for each method. The DR was calculated based on the 99th percentile threshold of the normal traffic scores. Blue and red bars represent normal and abnormal traffic flows, respectively.

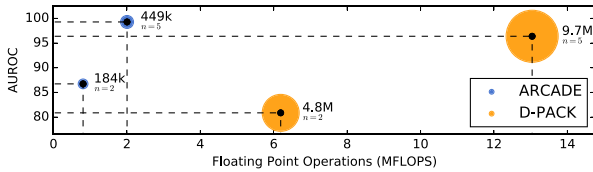


Fig. 7. Comparison between efficiency, effectiveness, and model size of ARCADE and D-PACK. We report AUROC (%) vs. floating-point operations (FLOPS) required for a single forward pass is reported with $n \in \{2, 5\}$. The size of each circle corresponds to the model size (number of parameters). ARCADE achieves higher AUROC with approximately 20 times fewer parameters than D-PACK.

NVIDIA GeForce RTX 3090 GPU, and 128 GB RAM as a cloud device. Detection speed experiments were conducted with and without GPU support to account for the fact that edge (and sometimes fog) nodes may not have a GPU device, as is the case with the Raspberry Pi 4 and the UP Xtreme board. The NVIDIA Jetson Xavier NX and the personal computer were given a GPU warm-up stage of 5 seconds immediately before starting the experiment. The mean amount of processed flows per second was computed given ten runs. All experiments were implemented in Python 3.8 PyTorch version 1.8 without any improvement to speed up inference. Table IX present the detection speed results with $n = 2$. The results show that ARCADE outperformed D-PACK in all environments, with ARCADE being approximately 8, 3, 2.8, 2, 2.16 times faster on the Raspberry Pi 4, UP Xtreme, NVIDIA Jetson, Threadripper CPU, and RTX 3090 GPU, respectively. ARCADE can process over 1.9M flows per second on the RTX 3090 GPU. The definition of an “optimal model” in an online network detection scenario cannot be well-defined

TABLE IX
MEAN DETECTION SPEED COMPARISON BETWEEN
ARCADE AND D-PACK

Device	Detection Speed (flows/s)	
	D-PACK	ARCADE
Raspberry Pi 4 Model B	79	661
UP Xtreme WHL17-A20-16064	4,120	12,471
NVIDIA Jetson Xavier NX	23,270	66,737
Ryzen Threadripper 3970X	22,659	47,478
NVIDIA GeForce RTX 3090	888,492	1,926,169

since there is a clear trade-off between the model's effectiveness and complexity. In this sense, the proposed model can be easily adapted by changing the number of layers and channels, together with the input size, to better suit the needs of a particular environment given its processing power capabilities.

VI. CONCLUSION

In this work, we introduced ARCADE, a novel unsupervised DL method for network anomaly detection that automatically builds the normal traffic profile based on raw network bytes as input without human intervention for feature engineering. ARCADE is composed of a 1D-CNN AE that is trained exclusively on normal network traffic flows and regularized through a WGAN-GP adversarial strategy. We experimentally demonstrated that the proposed adversarial regularization improves the performance of the AE. The proposed adversarial regularization strategy can be applied to any AE independently of its model architecture, and it can also be applied to other anomaly detection tasks. Once the AE is trained on the normal

TABLE X
ENCODER, DECODER, AND CRITIC ARCHITECTURE

Layer	K, S, P ¹	Output	Parameters
Encoder			
Input	—	1 × 200	—
Convolution	4, 2, 1	16 × 100	64
Batch Normalization	—	—	32
Leaky ReLU	—	—	—
Convolution	4, 2, 1	32 × 50	2,048
Batch Normalization	—	—	64
Leaky ReLU	—	—	—
Convolution	4, 2, 1	64 × 25	8,192
Batch Normalization	—	—	128
Leaky ReLU	—	—	—
Linear	—	50	80,000
Mean	—	—	90,528
Decoder			
Input	—	50	—
Linear	—	64 × 25	80,000
Batch Normalization	—	—	3,200
ReLU	—	—	—
Transposed Convolution	4, 2, 1	32 × 50	8,192
Batch Normalization	—	—	64
ReLU	—	—	—
Transposed Convolution	4, 2, 1	16 × 100	2,048
Batch Normalization	—	—	32
ReLU	—	—	—
Transposed Convolution	4, 2, 1	1 × 200	64
Sigmoid	—	—	—
Mean	—	—	93,600
Critic			
Input	—	1 × 200	—
Convolution	4, 2, 1	16 × 100	64
Layer Normalization	—	—	3,200
Leaky ReLU	—	—	—
Convolution	4, 2, 1	32 × 50	2,048
Layer Normalization	—	—	3,200
Leaky ReLU	—	—	—
Convolution	4, 2, 1	64 × 25	8,192
Layer Normalization	—	—	3,200
Leaky ReLU	—	—	—
Linear	—	50	80,000
Layer Normalization	—	—	100
Leaky ReLU	—	—	—
Linear	—	1	50
Mean	—	—	100,105

¹Kernel, Stride, Padding

TABLE XI
SPECIFICATIONS OF CONSIDERED ENVIRONMENTS FOR DETECTION SPEED EXPERIMENTS

Device	OS ¹	Mem. ²	CPU	GPU
Raspberry Pi 4B	Pi OS Lite 32-bit	4GB	Cortex-A72	—
UP Xtreme WHL17	Ubuntu 20.10 64-bit	16GB	Intel Core i7-8665UE	—
Jetson Xavier NX	Jetson Linux R32.7.1 64-bit	8GB	Carmel ARM	NVIDIA Volta with 384 CUDA cores and 48 tensor cores
Desktop 1	Ubuntu 20.10 64-bit	128GB	AMD Ryzen Threadripper 3970X	—
Desktop 2	Ubuntu 20.10 64-bit	128GB	AMD Ryzen Threadripper 3970X	NVIDIA RTX 3090 with 10,496 CUDA cores and 328 tensor cores

network traffic using the proposed approach, ARCADE can effectively detect unseen network traffic flows from attacks and malware. Our results suggest that even considering only

two initial packets of a network flow as input, ARCADE can detect most of the malicious traffic with nearly 100% F1-score, except for HTTP DoS and DDoS, where 68.70% and 66.61% F1-scores were obtained. While considering five packets, ARCADE achieved 91.95% and 93.19% F1-scores for HTTP DoS and DDoS, respectively. Experiments show that the proposed AE model is 20 times smaller than baselines and still presents significant improvement in accuracy, model complexity, and detection speed.

APPENDIX

See Tables X and XI.

REFERENCES

- [1] "Cisco annual Internet report (2018–2023)," Cisco, San Jose, CA, USA, White Paper, 2020.
- [2] H.-J. Liao, C.-H. R. Lin, Y.-C. Lin, and K.-Y. Tung, "Intrusion detection system: A comprehensive review," *J. Netw. Comput. Appl.*, vol. 36, no. 1, pp. 16–24, 2013.
- [3] J. V. V. Silva, N. R. de Oliveira, D. S. Medeiros, M. A. Lopez, and D. M. Mattos, "A statistical analysis of intrinsic bias of network security datasets for training machine learning mechanisms," *Ann. Telecommun.*, vol. 77, pp. 555–571, Feb. 2022.
- [4] Z. Ahmad, A. S. Khan, C. W. Shiang, J. Abdullah, and F. Ahmad, "Network intrusion detection system: A systematic study of machine learning and deep learning approaches," *Trans. Emerg. Telecommun. Technol.*, vol. 32, no. 1, 2021, Art. no. e4150.
- [5] T. Truong-Huu et al., "An empirical study on unsupervised network anomaly detection using generative adversarial networks," in *Proc. 1st ACM Workshop Security Privacy Artif. Intell.*, 2020, pp. 20–29.
- [6] R.-H. Hwang, M.-C. Peng, C.-W. Huang, P.-C. Lin, and V.-L. Nguyen, "An unsupervised deep learning model for early network traffic anomaly detection," *IEEE Access*, vol. 8, pp. 30387–30399, 2020.
- [7] M. Rudolph, B. Wandt, and B. Rosenhahn, "Same same but DifferNet: Semi-supervised defect detection with normalizing flows," in *Proc. IEEE/CVF Winter Conf. Appl. Comput. Vis.*, 2021, pp. 1907–1916.
- [8] D. Gong et al., "Memorizing normality to detect anomaly: Memory-augmented deep autoencoder for unsupervised anomaly detection," in *Proc. IEEE/CVF Int. Conf. Comput. Vis.*, 2019, pp. 1705–1714.
- [9] S. Zhai, Y. Cheng, W. Lu, and Z. Zhang, "Deep structured energy based models for anomaly detection," in *Proc. Int. Conf. Mach. Learn.*, 2016, pp. 1100–1109.
- [10] A. Radford, L. Metz, and S. Chintala, "Unsupervised representation learning with deep convolutional generative adversarial networks," 2015, *arXiv:1511.06434*.
- [11] I. Goodfellow et al., "Generative adversarial nets," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 27, 2014, pp. 1–9.
- [12] M. Arjovsky, S. Chintala, and L. Bottou, "Wasserstein generative adversarial networks," in *Proc. Int. Conf. Mach. Learn.*, 2017, pp. 214–223.
- [13] I. Gulrajani, F. Ahmed, M. Arjovsky, V. Dumoulin, and A. C. Courville, "Improved training of Wasserstein GANs," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 30, 2017, pp. 5769–5779.
- [14] L. Vu, C. T. Bui, and Q. U. Nguyen, "A deep learning based method for handling imbalanced problem in network traffic classification," in *Proc. 8th Int. Symp. Inf. Commun. Technol.*, 2017, pp. 333–339.
- [15] R. Doriguzzi-Corin, S. Millar, S. Scott-Hayward, J. Martinez-del Rincon, and D. Siracusa, "Lucid: A practical, lightweight deep learning solution for ddos attack detection," *IEEE Trans. Netw. Service Manage.*, vol. 17, no. 2, pp. 876–889, Jun. 2020.
- [16] W. Wang et al., "Hast-IDS: Learning hierarchical spatial-temporal features using deep neural networks to improve intrusion detection," *IEEE Access*, vol. 6, pp. 1792–1806, 2017.
- [17] W. Wang, M. Zhu, X. Zeng, X. Ye, and Y. Sheng, "Malware traffic classification using convolutional neural network for representation learning," in *Proc. Int. Conf. Inf. Netw.*, 2017, pp. 712–717.
- [18] Y. Yu, J. Long, and Z. Cai, "Network intrusion detection through stacking dilated convolutional autoencoders," *Security and Communication Networks*, vol. 2017, Nov. 2017, Art. no. 4184196.

- [19] W. Wang, M. Zhu, J. Wang, X. Zeng, and Z. Yang, "End-to-end encrypted traffic classification with one-dimensional convolution neural networks," in *Proc. Int. Conf. Intell. Security Informat.*, 2017, pp. 43–48.
- [20] G. Aceto, D. Ciunzo, A. Montieri, and A. Pescapé, "Mobile encrypted traffic classification using deep learning: Experimental evaluation, lessons learned, and challenges," *IEEE Trans. Netw. Service Manage.*, vol. 16, no. 2, pp. 445–458, Jun. 2019.
- [21] M. Lotfollahi, M. J. Siavoshani, R. S. H. Zade, and M. Saberian, "Deep packet: A novel approach for encrypted traffic classification using deep learning," *Soft Comput.*, vol. 24, no. 3, pp. 1999–2012, 2020.
- [22] T. Ahmad, D. Truscan, J. Vain, and I. Porres, "Early detection of network attacks using deep learning," 2022, *arXiv:2201.11628*.
- [23] P. Bergmann, S. Löwe, M. Fauser, D. Sattlegger, and C. Steger, "Improving unsupervised defect segmentation by applying structural similarity to autoencoders," 2018, *arXiv:1807.02011*.
- [24] Z. Wang, A. C. Bovik, H. R. Sheikh, and E. P. Simoncelli, "Image quality assessment: From error visibility to structural similarity," *IEEE Trans. Image Process.*, vol. 13, pp. 600–612, 2004.
- [25] G. Pang, C. Shen, L. Cao, and A. V. D. Hengel, "Deep learning for anomaly detection: A review," *ACM Comput. Surveys*, vol. 54, no. 2, pp. 1–38, 2021.
- [26] T. Schlegl, P. Seeßböck, S. M. Waldstein, U. Schmidt-Erfurth, and G. Langs, "Unsupervised anomaly detection with generative adversarial networks to guide marker discovery," in *Proc. Int. Conf. Inf. Process. Med. Imag.*, 2017, pp. 146–157.
- [27] H. Zenati, C. S. Foo, B. Lecouat, G. Manek, and V. R. Chandrasekhar, "Efficient GAN-based anomaly detection," 2018, *arXiv:1802.06222*.
- [28] S. Akcay, A. Atapour-Abarghouei, and T. P. Breckon, "Ganomaly: Semi-supervised anomaly detection via adversarial training," in *Proc. Asian Conf. Comput. Vis.*, 2018, pp. 622–637.
- [29] J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li, and L. Fei-Fei, "ImageNet: A large-scale hierarchical image database," in *Proc. Int. Conf. Comput. Vis. Pattern Recognit.*, 2009, pp. 248–255.
- [30] Z. Xiao, Q. Yan, and Y. Amit, "Do we really need to learn representations from in-domain data for outlier detection?" 2021, *arXiv:2105.09270*.
- [31] L. Bergman, N. Cohen, and Y. Hoshen, "Deep nearest neighbor anomaly detection," 2020, *arXiv:2002.10445*.
- [32] T. Reiss, N. Cohen, L. Bergman, and Y. Hoshen, "Panda: Adapting pretrained features for anomaly detection and segmentation," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2021, pp. 2806–2814.
- [33] A. Shiravi, H. Shiravi, M. Tavallaei, and A. A. Ghorbani, "Toward developing a systematic approach to generate benchmark datasets for intrusion detection," *comput. Security*, vol. 31, no. 3, pp. 357–374, 2012.
- [34] C. D. McDermott, F. Majdani, and A. V. Petrovski, "BotNet detection in the Internet of Things using deep learning approaches," in *Proc. Int. Joint Conf. Neural Netw.*, 2018, pp. 1–8.
- [35] B. Schölkopf, J. C. Platt, J. Shawe-Taylor, A. J. Smola, and R. C. Williamson, "Estimating the support of a high-dimensional distribution," *Neural Comput.*, vol. 13, no. 7, pp. 1443–1471, 2001.
- [36] F. T. Liu, K. M. Ting, and Z.-H. Zhou, "Isolation forest," in *Proc. Int. Conf. Data Min.*, 2008, pp. 413–422.
- [37] S. Kastyulin, D. Zakirov, and D. Prokopenko, "PyTorch image quality: Metrics and measure for image quality assessment." 2019. [Online]. Available: <https://github.com/photosynthesis-team/piq>
- [38] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," 2014, *arXiv:1412.6980*.



Willian Tessaro Lunardi (Member, IEEE) received the Ph.D. degree in computer science from the University of Luxembourg. He is a Senior Research Scientist with the Secure Systems Research Centre, Technology Innovation Institute, Abu Dhabi, UAE. He has published over 25 research papers in international scientific journals, conferences, and book chapters. His main area of research is machine learning and combinatorial optimization. He is currently working on machine learning for network security, physical layer security, and jamming detection.



Martin Andreoni Lopez (Member, IEEE) graduated as an electronic engineer from the Universidad Nacional de San Juan, Argentina, in 2011, the master's degree in electrical engineering from the Federal University of Rio de Janeiro (COPPE/UFRJ) in 2014, and the Doctoral degree from the Teleinformatics and Automation Group, COPPE/UFRJ and the Phare team of Laboratoire d'Informatique Paris VI, Sorbonne Université, France, in 2018. He is a Network Security Researcher with the Secure System Research Center, Technology Innovation Institute, Abu Dhabi, UAE. He was a Researcher with Samsung R&D Institute Brazil. He has coauthored several publications and patents in security, virtualization, traffic analysis, and big data.



Jean-Pierre Giacalone received the engineering degree from the École nationale supérieure d'électrotechnique, d'électronique, d'informatique, d'hydraulique et des télécommunications, Toulouse, France. He is the Vice President of Secure Communications Engineering with the Secure Systems Research Centre, Technology Innovation Institute, Abu Dhabi, UAE. He is responsible for researching secure communications, focusing on improving the resilience of cyber-physical and autonomous systems. He has worked as an Expert in software architecture for advanced driving assistance systems with Renault and as a Principal Engineer and Architect within Intel's Mobile Systems Technologies Group. He holds 19 patents and has coauthored 15 research papers accepted for publication in international journals and conference proceedings.